A SHORT INTRODUCTION TO BOOSTING

By Yoav Freund and Robert E. Schapire

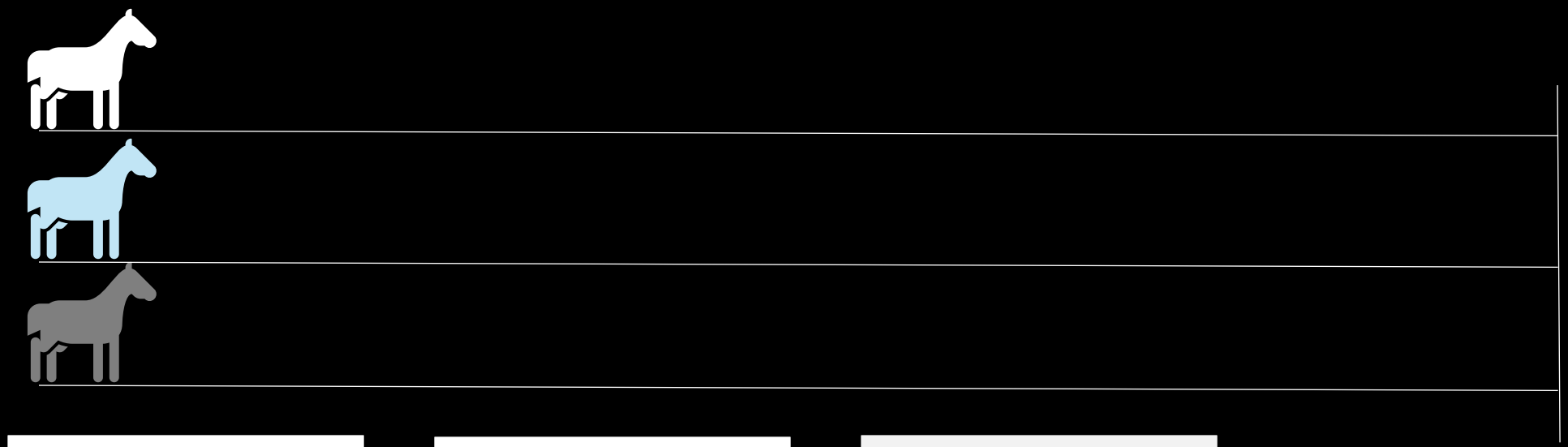
Journal of Japanese Society For Artificial Intellligence, September 1999

presented by Surya Setty

DSC 180A Paper Presentation



RULES OF THUMB: HORSE RACE BETTING



I bet on the horse that won the last race usually!

I bet on the horse that has the best odds!

I bet on the horse that's the youngest!



The core idea

AdaBoost combines many simple models into one stronger predictor.

1 Start simple

Train a weak learner—always generates a hypotheses with a weak edge for any distribution

2 Reweight examples

Give more influence to training examples that the current learner misclassifies.

3 Combine learners

Repeat, then make a weighted vote: stronger learners receive larger weights.

Creates a strong learner- can generate a hypothesis with an arbitrarily low error rate given sufficient data

Sequential algorithm

Error of a Weak Learner: $$\epsilon_t = \Pr_{i \sim D_t}\left[h_t(x_i) \neq y_i\right] = \sum_{i:h_t(x_i)\neq y_i} D_t(i).$$

probability that the weak learner is wrong on a random point drawn using distribution of weights D

Sum up the weights where prediction is wrong

Notation

Training data:

$$X = (x_i, y_i)_{i=1}^{n}, y_i \in Y = \{1, 1\}$$

Domain X and Labels Y

Initialize equal example weights:

$$D_0(i) = \frac{1}{n} \forall \; i = 1, 2, ...n$$

Weights array of length n

At each boosting round t

Fit a weak classifier $h_t(x)$ using the current weights. Then, update the weights.

The weights describe which training examples to focus on. Adaboost focuses on the hardest examples.

Round t: measure the learner error

$$\epsilon_t = \Pr_{i \sim D_t}\left[h_t(x_i) \neq y_i\right]$$

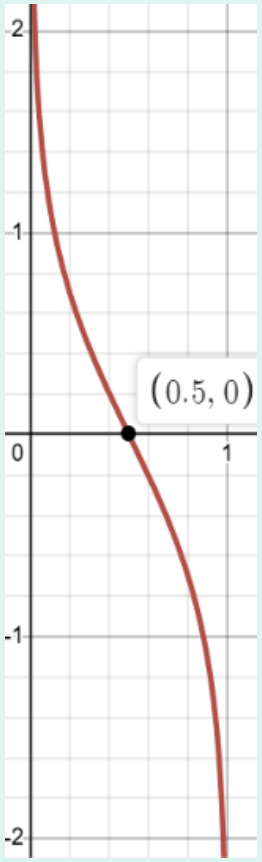
$$\epsilon_t = \sum_{i=[h_t(x_i) \neq y_i]} D_t(i)$$

Weighted error: add the weights of examples classified incorrectly.

$$\alpha_t = \frac{1}{2} ln(\frac{1-\epsilon_t}{\epsilon_t})$$

Learner weight: lower error gives a larger vote in the final model.

*have some handling for cases where alpha is very small



Distribution of α_t

A more accurate weak learner gets a larger positive vote.

Update example weights

$$
\begin{aligned}
D_{t+1}(i) &= \frac{D_t(i)}{Z_t} \times \begin{cases} e^{-\alpha_t} & \text{if } h_t(x_i) = y_i \\ e^{\alpha_t} & \text{if } h_t(x_i) \neq y_i \end{cases} \\
&= \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}
\end{aligned}
$$

Z is a normalization term

Make the weight of the example bigger if it is guessed incorrectly, or else make the example weight smaller in the next round

Correctly classified

Multiply by $\exp(-\alpha_t)$

Misclassified

Multiply by $\exp(+\alpha_t)$

Final Hypothesis

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

For each point, weigh the result of the weak learner on that point and return if the predicted point is in the positive or negative class. The middle term can be interpreted as confidence.

Final Algorithm

ITHM

Input:
  Training set S = {(x_1, y_1), …, (x_N, y_N)}, where y_i ∈ {-1, +1}
  Number of rounds T
  Base (weak) learner algorithm WeakLearn

Initialize:
  For i = 1 to N:
    w_i ← 1 / N // uniform sample weights

For t = 1 to T:

  1. Train weak learner h_t using S with weights w
     h_t ← WeakLearn(S, w)

  2. Compute weighted error
     ε_t ← Σ_{i : h_t(x_i) ≠ y_i} w_i

  3. Compute learner weight
     α_t ← 0.5 * ln((1 - ε_t) / ε_t)

  4. Update sample weights
     For i = 1 to N:
       w_i ← w_i * exp(-α_t * y_i * h_t(x_i))

  5. Normalize weights
     Z_t ← Σ_i w_i
     For i = 1 to N:
       w_i ← w_i / Z_t

Output:
  Final classifier
    H(x) = sign(Σ_{t=1}^{T} α_t * h_t(x))

Theorem 1: Training Error bounds

Define the Error of A Weak Learner, (how much better than random in binary problems)

$$\epsilon_t(h_t) = \frac{1}{2} - \gamma_t$$

Freund and Schapire found training error (fraction of mistakes on training set) is bounded by

$$\prod_t \left[2\sqrt{\epsilon_t(1-\epsilon_t)}\right] = \prod_t \sqrt{1-4\gamma_t^2} \leq \exp\left(-2\sum_t \gamma_t^2\right).$$

If each hypothesis is slightly better than random, then the training error drops exponentially fast

$\gamma_t \geq \gamma$ $\gamma > 0$

AdaBoost adapts to the error rates of individual weak hypotheses.

Theorem 2: Generalization/Test Error

sample size - m
VC dimension – d
Boosting rounds - T

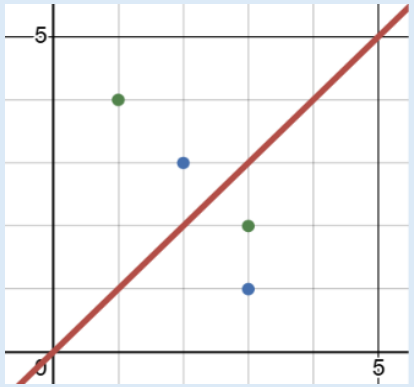
Freund and Schapire found test error (fraction of mistakes on test set) is bounded by

$$\underbrace{\hat{\Pr}\left[H(x) \neq y\right]}_{\text{unweighted train error}} + \underbrace{\tilde{O}\left(\sqrt{\frac{Td}{m}}\right)}_{\text{ensemble complexity}}$$

Key: model complexity has to be balanced by having enough data

Shattering -> a model can shatter this number of points if it can separate them regardless of their combination of labels

VC dimension -> number of points it can shatter at maximum



Linear 2D model has d = 3

Contradiction

$$\hat{\Pr}\left[H(x) \neq y\right] + \tilde{O}\left(\sqrt{\frac{Td}{m}}\right)$$

If generalization error can get higher by the amount of rounds, does that imply AdaBoost overfits? Not necessarily.

This is explained by margins.

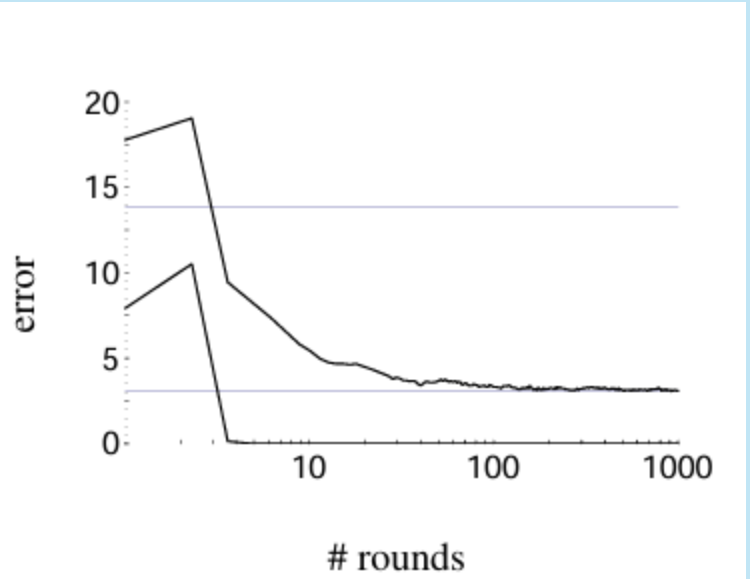


Figure 2: Error curves from boosting C4.5 on letter dataset

Margins

Define a margin of an example (x, y) as

$$\frac{y \sum_t \alpha_t h_t(x)}{\sum_t \alpha_t}$$

Large margin = High confidence

Schapire et al. found larger margins on training set found a different bound on generalization error not defined over T for theta > 0 with high probability

$$\hat{\Pr}\left[\text{margin}(x, y) \leq \theta\right] + \tilde{O}\left(\sqrt{\frac{d}{m\theta^2}}\right)$$

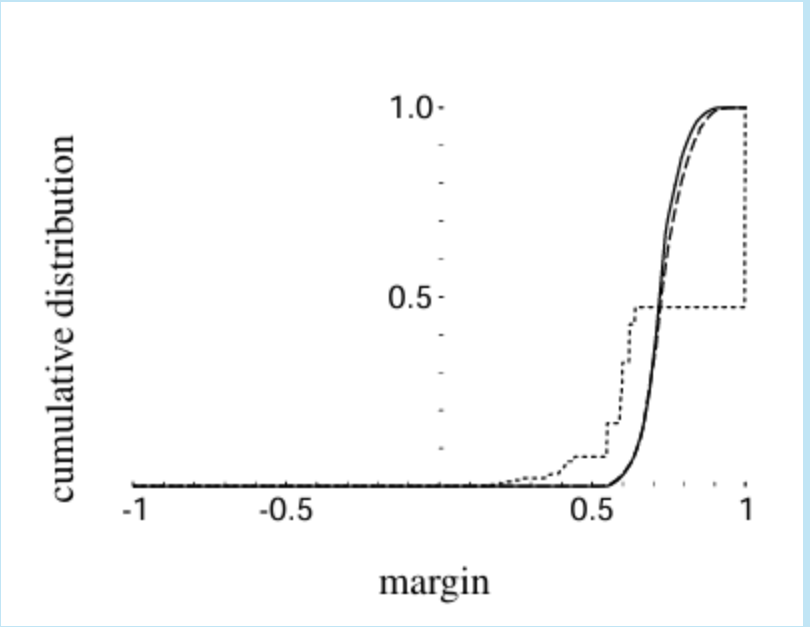


Figure 2: CDF of margins on training examples after t=5, 100, and 1000 curves from boosting C4.5 on letter dataset (short-dash, long-dash, and solid curves)

Boosting, conclusions

Boosting aggressively concentrates on the examples with the smallest margins, reducing the margins even after train error reaches 0, which can reduce test error.

Game Theory: Boosting repeatedly plays the same game, where AdaBoost can be seen as a general algorithm to roughly solve a game.

Relationship to SVMs

Suppose we have zero training error, and want to maximize the minimum margin of any training example.

$$\max_{\boldsymbol{\alpha}} \min_{i} \frac{(\boldsymbol{\alpha} \cdot \mathbf{h}(x_i))y_i}{||\boldsymbol{\alpha}||\ ||\mathbf{h}(x_i)||}$$

$$h(x) =< h_1(x), h_2(x), ..., h_N(x) >$$

$$\alpha =< \alpha_1, \alpha_2, ..., \alpha_N(x) >$$

Boosting

$$||\boldsymbol{\alpha}||_1 \doteq \sum_t |\alpha_t|, \qquad ||\mathbf{h}(x)||_\infty \doteq \max_t |h_t(x)| \ .$$

1 norm and infinity Norm

SVMs

$$||\boldsymbol{\alpha}||_2 \doteq \sqrt{\sum_t \alpha_t^2}, \qquad ||\mathbf{h}(x)||_2 \doteq \sqrt{\sum_t h_t(x)^2} \ .$$

Euclidean norms

Differences between SVM and Boosting

In high dimensional spaces, the difference between norms is large (such as SVM and Boosting) and leads to large differences in the values of margins

SVM – Quadratic Programming
AdaBoost – Linear Programming
(Game Theory Example)

Searching Efficiently:
SVM – Kernel Method for Efficient Inner Products
Boosting – Greedy Search using the Weak Learner and updating it

Choice of Kernel/Learner matters significantly

Multiclass Expansion

If you have a strong weak learner (epsilon < 0.5), AdaBoost.M1 is a straightforward generalization

(Freund and Schapire, A decision-theoretic generalization of online-learning and an application to boosting, 1997)

Reduce the multiclass to a larger binary, such as AdaBoost.MH. Asks a set of binary questions: "For x, is the correct label y or one of the other labels?"

AdaBoost.M2 asks "For example x with correct label y and an incorrect label y', is the correct label y or y'?"

(Schapire and Singer, Improved boosting algorithms using confidence-rated predictions, 1998)

Advantages and Disadvantages

Advantages

- ❑ Easy to program and simple with one parameter T
- ❑ Next to no requirements on weak learner, just need to be better than random
- ❑ High ability to identify outliers

Disadvantages

- ❑ Performance dependent on data and weak learner
- ❑ Fails to work well without enough data, especially complex weak hypotheses, extremely weak hypotheses

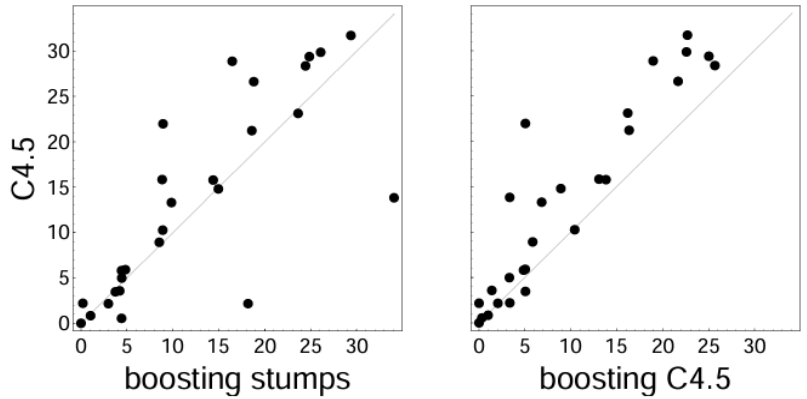
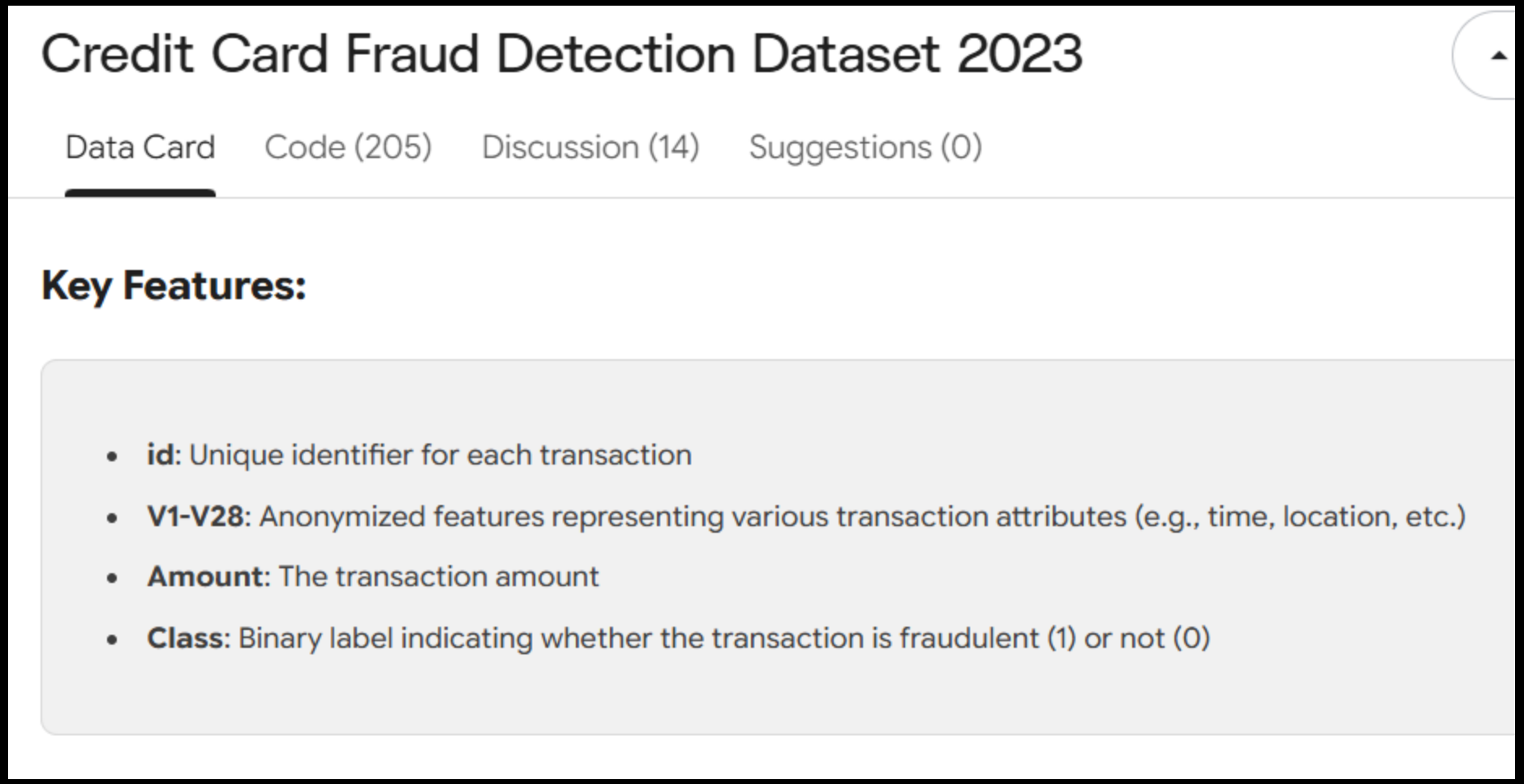


Figure 3: C4.5 versus boosting stumps and boosting C4.5 using test error. Points in the top left represent where the x variable performs better.



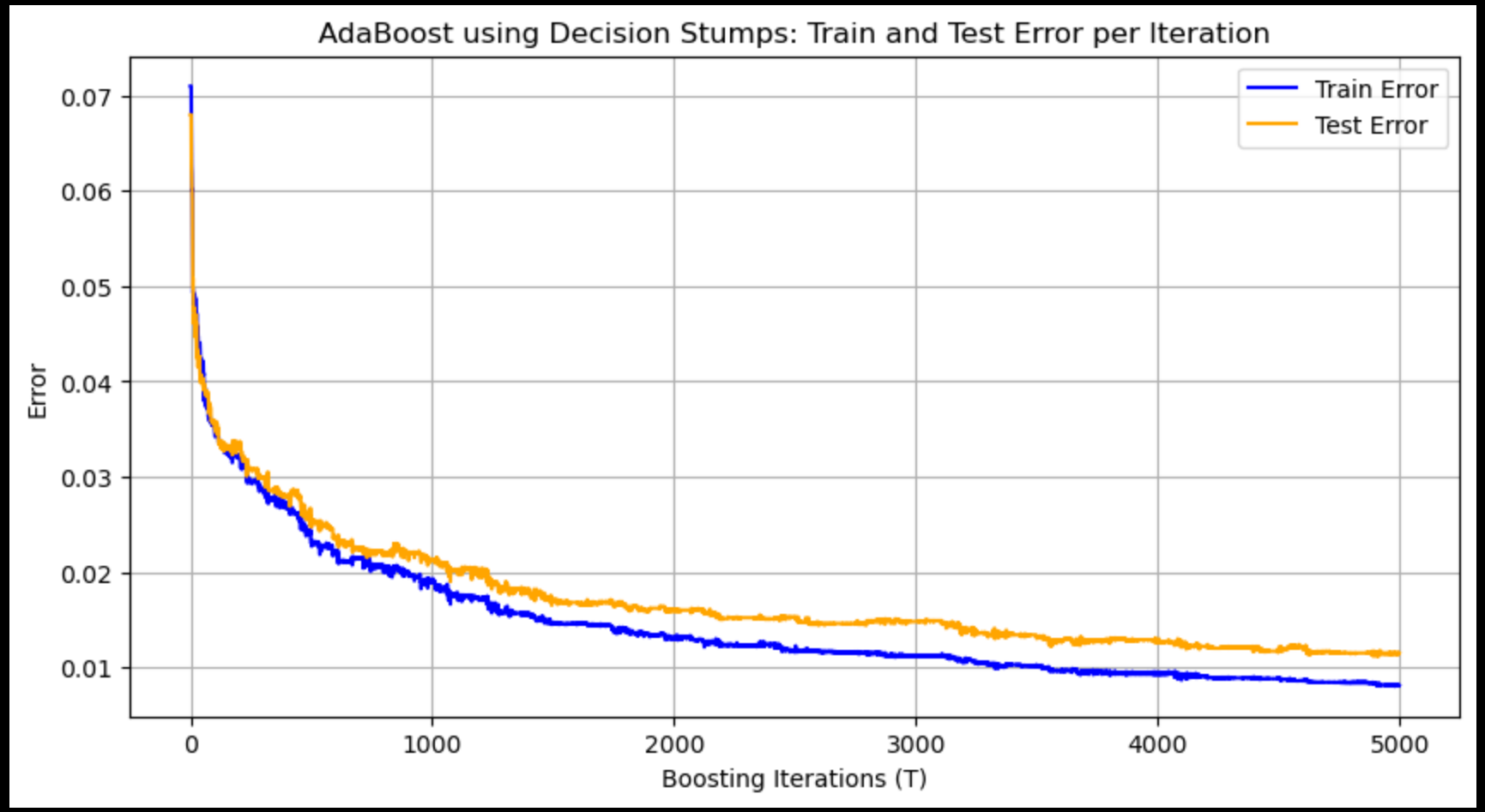
CREDIT CARD FRAUD 2023

Source: Kaggle

Popular Dataset on Kaggle for Classification, Balanced classes may indicate synthetic data

Many rows (550k) and features (28)

ANONYMIZED CREDIT CARD DATA FROM EUROPEAN CARDHOLDERS IN 2023



ACCURACY

Note that this is a sequential algorithm, meaning it cannot take advantage of parallelization from GPUs.

I ran a sampled smaller version (50k observations) .

Running this on my laptop took 8 minutes and 48 seconds.

Breast Cancer Wisconsin Performance

Breast Cancer Wisconsin is a popular classification dataset with 569 tumors (212 malignant) and 30 features.

Method	Weak Learner	Train Error	Test Error	Train (s)	One Prediction (μs)
T=10	Decision Stump	0.022 ± 0.0018	0.049 ± 0.0056	0.204	0.042
T=100	Decision Stump	0 ± 0	0.029 ± 0.0038	2.18	0.13
T=10	Decision Tree (depth=3)	0 ± 0	0.042 ± 0.0047	0.309	0.019
T=100	Decision Tree (depth=3)	0 ± 0	0.031 ± 0.0042	3.09	0.22

Conclusions

Applications

- Significantly improves the performance of a weak learner
- Boosting for text categorization

Extensions

- Friedman's Gentle AdaBoost emphasizes outliers less
- Freund's BrownBoost de-emphasizes outliers when they are too hard to classify correctly

Alternative Interpretation

- Schapire and Singer generalize boosting by interpreting it as a form of gradient descent associating a cost with each example based on current margin, and AdaBoost operation as coordinate-wise gradient descent in realm of linear classifiers
- Design boosting to learn rule lists